I have a simple Customer class in c# that has the following member variables:
- CustomerID (GUID)
- CustomerFirstname (string)
- CustomerSurname (string)
- CustomerBirthdate (DateOnly)
- CustomerEmail (string)
- CustomerPhone (string)

When creating a customer using the constructor, these fields (aside from the generated UID) are validated using a simple validator class. These are relatively simplistic validation checks, using Regex to check the strings and bounding dates to compare the birthdate to.

I would like some help designing unit tests for these validation methods please. Let's begin with a planning phase, and scope out the tests we will need to design. The following Regex patterns are what will be used, so we can look at these and assess what we will reasonably need to test:
- NamePattern: ^[A-Za-z]+$
- PhonePattern: ^(?:\+44|0)\d{9,10}$
- EmailPattern: ^[A-Za-z0-9.+_-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)+$

So name should be alphabetic only, phone should start with 0 or +44 (UK code) and have 9 or 10 following digits, and email should be alphanumeric (with limited special characters), followed by an @ symbol, then more

ChatGPT Get Plus ×

Great—starting with a **planning phase** is exactly the right approach. Below is a structured way to scope and design the unit tests you'll need, focusing on **validation intent**, **boundary conditions**, and **failure modes**, rather than just "happy path vs broken".

I'll assume:

- Each validation method returns `bool` **or**
- Throws a specific exception (e.g. `ArgumentException`)

  Either approach works for the same test plan.

---

1. High-level testing strategy

For each validated field, we want to cover:

1. **Valid inputs**
   Values that *should* pass according to business rules and regex intent.
2. **Invalid inputs (format)**
   Values that violate the regex pattern.
3. **Boundary conditions**
   Inputs that sit exactly on min/max limits.
4. **Null / empty / whitespace**
   Even if regex fails these implicitly, tests should make expectations explicit.
5. **Semantic edge cases**
   Inputs that technically match regex but may be questionable in real-world use.

This keeps tests **expressive**, **maintainable**, and **future-proof**.

---

2. Name validation (`^[A-Za-z]+$`)

Validation intent

- Alphabetic characters only
- At least 1 character
- No spaces, hyphens, accents, or apostrophes

Test categories

✅ Valid cases

These ensure the regex works as intended.

- Single-letter name (`"A"`)
- Typical name (`"John"`)
- Long name (`"Christopher"`)

❌ Invalid – forbidden characters

These catch incorrect formats.

- Contains spaces (`"John Smith"`)
- Contains hyphen (`"Anne-Marie"`)

- Contains apostrophe (`"O'Neil"`)
- Contains numbers (`"John1"`)
- Contains symbols (`"John!"`)
- Contains accented characters (`"José"`)

> ⚠️ Even if accented names are *realistic*, your regex explicitly rejects them—tests should reflect that current design.

❌ **Invalid – empty / null**
Important for constructor robustness.

- Empty string (`""`)
- Whitespace only (`" "`)
- `null`

---

3. Phone number validation (`^(?:\+44|0)\d{9,10}$`)

Validation intent

- UK phone numbers only
- Must start with `0` **or** `+44`
- Followed by **9 or 10 digits**
- No spaces or separators

Test categories

✅ **Valid cases**
Cover both allowed prefixes and both lengths.

- `0` prefix with 9 digits
- `0` prefix with 10 digits
- `+44` prefix with 9 digits
- `+44` prefix with 10 digits

These form your **core acceptance tests**.

❌ **Invalid – prefix issues**

- Starts with `44` (missing `+`)
- Starts with `+1` or other country code
- Starts with neither `0` nor `+44`

❌ **Invalid – length issues**

- Too short (8 digits after prefix)

- Too long (11+ digits after prefix)

❌ **Invalid – formatting characters**
Even though real numbers often include these, regex forbids them.

- Spaces (`"07123 456789"`)
- Dashes (`"07123-456789"`)
- Parentheses (`"(07123)456789"`)

❌ **Invalid – empty / null**

- Empty string
- Whitespace
- `null`

---

4. Email validation

`^[A-Za-z0-9.+_-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)+$`

Validation intent

- Alphanumeric local-part with limited symbols
- Exactly one `@`
- Domain must contain **at least one dot**
- No trailing dot
- No spaces

Test categories

✅ **Valid cases**
Cover common real-world emails.

- Simple email (`"test@example.com"`)
- Multiple subdomains (`"user@mail.example.co.uk"`)
- Allowed symbols (`"user.name+tag@example.com"`)

❌ **Invalid – local-part issues**

- Starts with `@`
- Contains spaces
- Contains invalid symbols (`"user!@example.com"`)

❌ **Invalid – domain issues**

- Missing dot (`"user@example"`)
- Trailing dot (`"user@example."`)
- Leading dot (`"user@.example.com"`)

- Consecutive dots (`"user@example..com"`)

❌ **Invalid – structure issues**
- Missing `@`
- Multiple `@` symbols

❌ **Invalid – empty / null**
- Empty string
- Whitespace
- `null`

---

5. Birthdate validation (non-regex)

Even though you didn't list rules explicitly, tests should cover:

Common constraints (typical)

- Birthdate **not in the future**
- Birthdate **not unreasonably far in the past** (e.g. > 120 years ago)

Test categories

✅ **Valid**
- Today minus 18 years
- Reasonable adult date
- Boundary valid date (e.g. exactly 120 years ago)

❌ **Invalid**
- Future date
- Exactly 1 day in the future
- Too far in the past
- `DateOnly.MinValue` (if applicable)

---

6. Constructor-level tests (integration-style)

Beyond individual validators, add **Customer constructor tests**:

- Creating a customer with **all valid values succeeds**
- Creating a customer with **each invalid field throws**
  - One test per field to isolate failures
- Constructor generates a **non-empty GUID**

These tests ensure validators are actually *wired up correctly*.

7. Test structure recommendation

For readability and maintainability:

- Use **parameterized tests** (e.g. `[Theory]` + `[InlineData]` in xUnit)
- Group tests by:
  - `NameValidationTests`
  - `PhoneValidationTests`
  - `EmailValidationTests`
  - `BirthdateValidationTests`
  - `CustomerConstructorTests`

Each test name should describe:

> **Given_X_When_Validated_Then_Y**

Example:

```
Name_WithNumbers_ShouldFailValidation
Phone_WithInvalidPrefix_ShouldThrow
Email_WithoutDomainDot_ShouldBeInvalid
```

---

8. Next step

If you'd like, next we can:

- Translate this plan into **xUnit / NUnit / MSTest examples**
- Design **parameterized test data tables**
- Discuss whether **exceptions vs result objects** are better for validation
- Review whether the regexes match real-world expectations

Just tell me where you'd like to go next 👍

You made an excellent point about the name regex limitations, I have since updated this:
^[A-Za-zÀ-ÿ-']+$
Just to check, does this appear to be more appropriate, allowing for hyphenated and apostrophe names, and accepting accented characters?

Short answer: **yes, this is a *much* more appropriate regex for real-world names**, and it's a solid improvement 👍
That said, there are a few nuances worth understanding so you know *exactly* what you're allowing.

---

1. What your updated regex does well

Your pattern

```
^[A-Za-zÀ-ÿ-']+$
```

✅ Improvements over the original

✔ **Accented characters**
- `À-ÿ` covers most **Latin-1 Supplement** letters
- This includes common accented characters:
  - é, è, ñ, ü, ç, å, ø, etc.
- This is a big win for international names

✔ **Hyphenated names**
- Allows names like:
  - `Anne-Marie`
  - `Jean-Luc`

✔ **Apostrophes**
- Allows names like:
  - `O'Connor`
  - `D'Arcy`

✔ **Still restrictive**
- No digits
- No whitespace
- No symbols beyond `-` and `'`
- Still enforces *at least one character*

Overall: **excellent balance between realism and validation strictness**.

---

2. Important limitations to be aware of (by design)

Even though this is better, there are still behaviors you should be consciously OK with.

⚠️ Allows “syntactically odd” but regex-valid names

The following would pass:

- `--`
- `''`
- `-'`
- `Jean--Luc`
- `O''Connor`
- `-John`
- `John-`

This happens because:

- `+` allows any combination of allowed characters
- There are no positional rules

Whether this matters depends on how strict you want to be.

---

3. If you want *slightly* stronger realism (optional)

If you want:

- Letters required at the start and end
- Hyphens/apostrophes only **between** letters

You could use:

```
^[A-Za-zÀ-ÿ]+([-' ][A-Za-zÀ-ÿ]+)*$
```

What this enforces:

- Must start with a letter
- Must end with a letter
- Hyphens/apostrophes can only appear **between name segments**
- Prevents double punctuation

Valid:

- `O'Connor`
- `Anne-Marie`
- `Jean-Luc`

Invalid:

- `--`

- `John-`
- `-John`
- `O''Connor`

> You can remove the space from `([-' ][A-Za-zÀ-ÿ]+)*` if you still want to forbid spaces.

---

4. Unicode-aware alternative (C#-specific)

Since you're in C#, you could consider a **Unicode-category-based** regex instead of hardcoding ranges:

```
^\p{L}+([-'’]\p{L}+)*$
```

Benefits:

- `\p{L}` = *any* Unicode letter
- Handles **non-Latin alphabets** (if that ever matters)
- More future-proof

---

5. Recommendation summary

If your app is:

- UK/EU-focused
- Latin alphabet only
- Simple validation needs

👉 **Your current regex is perfectly acceptable**

If you want better structure with minimal complexity

👉 Use the *letter–separator–letter* pattern

If you want maximum international correctness

👉 Use `\p{L}`

If you want, next we can:

- Update your **unit test plan** to reflect this new name behavior
- Compare validation strictness vs user experience

- Align name/email/phone validation philosophies for consistency

Your suggestion about the unicode-based regex is fascinating and worth exploring another time, but for now I have switched to using your suggested regex to prevent some of those edge cases. Now let us return to testing, let's translate the plan into xUnit examples please. I'd like to see a test you design so I can assess it's correctness and also check that it fits our conventions and standards well, please.

Excellent choice—that regex strikes a very sensible balance, and it *also* makes the tests more meaningful 👍
Let's now translate the **test plan into a concrete xUnit example**, focusing on **clarity, correctness, and convention** rather than sheer volume.

Below I'll:

1. State the **assumptions**
2. Show **one well-designed xUnit test class**
3. Explain **why it's structured this way**
4. Highlight **conventions and standards it follows**

---

1. Assumptions (explicit and testable)

I'll assume the following (adjust naming to match your codebase):

- You have a validator class, e.g.:

```C#
public static class CustomerValidator
{
    public static void ValidateFirstName(string firstName)
    {
        // throws ArgumentException if invalid
    }
}
```

- Validation methods **throw** on failure (preferred for constructor validation)

- Regex used:

```regex
^[A-Za-zÀ-ÿ]+([-' ][A-Za-zÀ-ÿ]+)*$
```

(no spaces if you removed them—I'll assume no spaces for this example)

---

2. Example xUnit test: First name validation

This test demonstrates:

- Parameterized testing
- Positive + negative coverage
- Clear naming
- Minimal noise

Test class

```C#
using System;
using Xunit;

public class CustomerFirstNameValidationTests
{
    [Theory]
    [InlineData("John")]
    [InlineData("Anne-Marie")]
    [InlineData("O'Connor")]
    [InlineData("Élodie")]
    [InlineData("Jean-Luc")]
    public void ValidateFirstName_WithValidNames_DoesNotThrow(string firstName)
    {
        // Act
        var exception = Record.Exception(() =>
            CustomerValidator.ValidateFirstName(firstName));

        // Assert
        Assert.Null(exception);
    }

    [Theory]
```

---

3. Why this test is well-designed

✅ Parameterized tests (`[Theory]`)

- Avoids repetitive boilerplate
- Easy to add new cases
- Keeps intent clear

✅ Clear test names

Follows the pattern:

```
MethodUnderTest_WithCondition_ExpectedOutcome
```

This makes failures self-explanatory in CI output.

✅ Focused responsibility

- One test class → one validation concern
- No coupling to other fields
- No constructor usage yet (that comes later)

✅ Uses `Record.Exception` for "does not throw"

This is preferable to:

```C#
CustomerValidator.ValidateFirstName(name); // hope nothing breaks
```

Because it **explicitly asserts success**, not just absence of failure.

✅ Includes edge cases that your new regex blocks

Examples like:

- `"-John"`
- `"O''Connor"`
- `"--"`

These ensure your improved regex stays enforced over time.

---

4. Conventions & standards checklist

- ✔ Uses AAA pattern (Arrange–Act–Assert)
- ✔ No magic strings in test logic
- ✔ Descriptive test data
- ✔ No reliance on regex details in tests

✔ Tests observable behavior, not implementation
✔ Fails loudly and meaningfully

---

5. Next logical steps

If you'd like, next we can:

- Write a **phone number validation test** (regex-heavy and very instructive)
- Write an **email validation test** (with boundary failures)
- Write **Customer constructor tests** that verify validators are called
- Refactor shared test data into `MemberData` or custom data attributes

Just tell me where you'd like to go next ✌️

Excellent, I'm very happy with those test cases you've generated for the names; please go ahead and proceed with the next test class.

Great — let's proceed with the **phone number validation tests** next.
I'll keep the same **style, conventions, and level of rigor** as the name tests so everything feels consistent across your test suite.

Assumptions (kept consistent)

- Validator method:

```csharp
public static void ValidatePhoneNumber(string phoneNumber)
```

- Throws `ArgumentException` when invalid
- Regex under test:

```regex
^(?:\+44|0)\d{9,10}$
```

This means:

- Prefix must be `0` or `+44`
- Followed by **9 or 10 digits**
- No spaces or separators

---

xUnit test class: Phone number validation

C#

```
using System;
using Xunit;

public class CustomerPhoneNumberValidationTests
{
    [Theory]
    [InlineData("07123456789")]        // 0 + 10 digits
    [InlineData("0712345678")]         // 0 + 9 digits
    [InlineData("+447123456789")]      // +44 + 10 digits
    [InlineData("+44712345678")]       // +44 + 9 digits
    public void ValidatePhoneNumber_WithValidPhoneNumbers_DoesNotThrow(string p
    {
        // Act
        var exception = Record.Exception(() =>
            CustomerValidator.ValidatePhoneNumber(phoneNumber));

        // Assert
        Assert.Null(exception);
    }

    [Theory]
    [InlineData("")]                   // empty
```

Why this test class is solid

✅ Explicit coverage of regex intent

Each category maps cleanly to a rule:

- Prefix validation
- Length validation
- Formatting rejection
- Null / empty handling

This makes the test suite act as **living documentation**.

✅ Boundary-aware

You explicitly test:

- Minimum length (9 digits)
- Maximum length (10 digits)
- One under / one over

This protects against future regex tweaks that accidentally loosen constraints.

✅ Real-world but policy-correct

Even though real users often enter:

- spaces
- dashes
- parentheses

Your regex *intentionally* rejects them — and the tests enforce that policy clearly.

✅ Consistent structure with name tests

- Same naming convention
- Same `[Theory]` + `[InlineData]`
- Same exception strategy
- Same `Record.Exception` pattern

This consistency pays off hugely as the test suite grows.

Subtle design win worth noting

You **did not** test the regex directly.
You tested **observable behavior** of the validator.

That means:

- You can change implementation (regex → library → API)
- Tests remain valid as long as behavior stays correct

Next suggested step

The natural progression is:

1. **Email validation tests** (most nuanced)

2. **Birthdate validation tests**
3. **Customer constructor tests** (integration-style)

If you'd like, next I can:

- Design the email tests (they're the trickiest and most instructive), or
- Jump straight to constructor-level tests to show how everything ties together

Just let me know 👍

Fantastic, let's get the email validation tests done then please.

Perfect — email validation is the richest one to test, so this is a great next step.
I'll keep everything **consistent with the previous test classes** and be very deliberate about *why* each case exists.

Assumptions (unchanged)

- Validator method:

```csharp
public static void ValidateEmail(string email)
```

- Throws `ArgumentException` when invalid
- Regex under test:

```regex
^[A-Za-z0-9.+_-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)+$
```

---

xUnit test class: Email validation

```csharp
using System;
using Xunit;
```

```
public class CustomerEmailValidationTests
{
    [Theory]
    [InlineData("test@example.com")]
    [InlineData("user.name@example.com")]
    [InlineData("user+tag@example.co.uk")]
    [InlineData("user_name@example-domain.com")]
    [InlineData("user.name+tag@example-domain.co.uk")]
    public void ValidateEmail_WithValidEmailAddresses_DoesNotThrow(string email
    {
        // Act
        var exception = Record.Exception(() =>
            CustomerValidator.ValidateEmail(email));

        // Assert
        Assert.Null(exception);
    }
```

Why these tests are particularly strong

✅ Mirrors the actual regex rules

Each invalid case corresponds directly to something your regex *forbids*:

- No spaces
- Exactly one `@`
- At least one dot in the domain
- No leading/trailing dots
- Limited character set

This ensures **zero mismatch** between validation policy and test expectations.

✅ Avoids false realism

Some email regexes try to be *too clever* and end up brittle.

You intentionally **do not** test:

- Quoted local parts (`"user"@example.com`)
- Unicode email addresses
- IP-based domains

Because your regex does not support them — and your tests correctly reflect that.

✅ Explicit null/empty coverage

Including `null`, empty, and whitespace in every validator test class is:

- Consistent
- Defensive
- Essential for constructor safety

✅ Consistent conventions across the suite

At this point, your test suite shows:

- Uniform structure
- Predictable naming
- Easy extensibility

That's exactly what you want in a professional codebase.

One subtle but important note

Your regex **allows**:

```
user..name@example.com
```

Because:

```
[A-Za-z0-9.+_-]+
```

If you *don't* want that, the tests are already positioned to catch a future change — you'd just move that case from "valid" to "invalid".

This is a good example of how tests **drive clarity of intent**.

What's left to complete the picture

You now have:

✔ Name validation tests
✔ Phone validation tests
✔ Email validation tests

Remaining pieces:

1. **Birthdate validation tests** (date logic & boundaries)

2. **Customer constructor tests** (end-to-end wiring)
3. Optional: **shared test data / theory data classes**

My recommendation: **constructor tests next**, because they validate everything you've built so far working together.

Shall we move on to those, or would you like to cover birthdate validation first?

Sure, we can do the constructor tests next then; I should also point out because I didn't before, that the GUID is generated when a customer is created and is therefore not passed to the constructor as a parameter; and, the CustomerGUID, CustomerFirstname, and CustomerSurname are all marked as required fields.

Excellent — that clarification is *exactly* what we need before writing constructor tests, and it aligns very well with the design you've been describing.

Constructor tests are where we **prove that all the individual validators are correctly composed** and that required fields are enforced.

What we are testing at constructor level

These are **integration-style unit tests** (still fast, still isolated):

Responsibilities of the constructor

1. Generates a **non-empty GUID**
2. Assigns all properties when inputs are valid
3. Enforces **required fields**
4. Invokes validation for:
   - First name
   - Surname
   - Birthdate
   - Email
   - Phone number
5. Fails fast when *any* validation fails

We **do not**:

- Re-test regex details
- Re-test individual validators
- Test internal implementation details

Assumed Customer constructor signature

I'll assume something like this (names adjusted to your description):

```C#
public class Customer
{
    public Guid CustomerID { get; }
    public string CustomerFirstname { get; }
    public string CustomerSurname { get; }
    public DateOnly CustomerBirthdate { get; }
    public string CustomerEmail { get; }
    public string CustomerPhone { get; }

    public Customer(
        string customerFirstname,
        string customerSurname,
        DateOnly customerBirthdate,
        string customerEmail,
        string customerPhone)
    {
        // validation + assignment + GUID generation
    }
}
```

---

xUnit test class: Customer constructor tests

This class demonstrates **best-practice constructor testing**.

```C#
using System;
using Xunit;

public class CustomerConstructorTests
{
    private static Customer CreateValidCustomer()
    {
        return new Customer(
            customerFirstname: "Anne-Marie",
            customerSurname: "O'Connor",
```

```
            customerBirthdate: DateOnly.FromDateTime(DateTime.Today.AddYears(-3
            customerEmail: "anne.oconnor@example.com",
            customerPhone: "07123456789");
    }

    [Fact]
    public void Constructor_WithValidArguments_CreatesCustomerSuccessfully()
    {
        // Act
        var customer = CreateValidCustomer();
```

Why this constructor test suite is well-designed

✅ Uses a *valid object factory*

`CreateValidCustomer()`:

- Prevents duplication
- Makes failures easier to diagnose
- Keeps tests focused on *what is invalid*

This is a **strong professional testing pattern**.

✅ Tests GUID behavior explicitly

You:

- Verify it's generated
- Verify uniqueness
- Avoid testing randomness or implementation details

✅ Required fields are tested clearly

First name & surname:

- `null`
- empty
- whitespace

This enforces your "required" constraint *independently* of regex rules.

✅ One failure per test

Each test invalidates **exactly one parameter**.
That's critical for:

- Readability
- Debugging
- Future refactoring

One optional refinement (advanced but clean)

If you want to be *very precise*, you could assert exception messages or parameter names:

```C#
var ex = Assert.Throws<ArgumentException>(() => ...);
Assert.Contains("CustomerFirstname", ex.Message);
```

This is optional, but helpful in larger domains.

Where you are now

At this point, you have:

✔ Field-level validator tests
✔ Constructor integration tests
✔ Clear required-field enforcement
✔ GUID generation verified

This is **very solid coverage** for a domain entity.

Final question (optional)

Would you like to:

- Add **birthdate validation unit tests** explicitly?
- Refactor common test data into reusable fixtures?
- Discuss whether `ArgumentException` vs `ArgumentNullException` vs `ArgumentOutOfRangeException` is worth differentiating?

Any of those would be a natural next refinement.

Let's finish up with the birth date tests. You actually got it spot on, 18 years and 120 years are exactly the parameters I've used to validate! Not sure how you did that, but happy coincidence!

😄 I'll take the happy coincidence — those limits are *very* common in well-designed domain models, so your instincts are spot on.

Let's finish cleanly with **birthdate validation tests**, keeping the same quality bar and conventions as everything else you now have.

What we are validating (made explicit)

Your birthdate rules:

- Customer must be **at least 18 years old**
- Customer must be **no more than 120 years old**
- Birthdate must **not be in the future**

These are *pure date logic rules*, so the tests focus heavily on **boundary conditions**.

Assumptions

- Validator method:

```csharp
public static void ValidateBirthdate(DateOnly birthdate)
```

- Throws `ArgumentException` when invalid
- Uses `DateOnly.FromDateTime(DateTime.Today)` internally (important for consistency)

---

xUnit test class: Birthdate validation

```csharp
using System;
using Xunit;
```

```
public class CustomerBirthdateValidationTests
{
    private static readonly DateOnly Today =
        DateOnly.FromDateTime(DateTime.Today);

    [Fact]
    public void ValidateBirthdate_WhenExactly18YearsOld_DoesNotThrow()
    {
        // Arrange
        var birthdate = Today.AddYears(-18);

        // Act
        var exception = Record.Exception(() =>
            CustomerValidator.ValidateBirthdate(birthdate));

        // Assert
        Assert.Null(exception);
    }
```

Why these tests are *exactly* right for date validation

✅ Boundary-first design

Every critical boundary is tested explicitly:

Case	Covered
Exactly 18	✅
Just under 18	✅
Exactly 120	✅
Just over 120	✅
Future date	✅

These are the tests most likely to catch regressions.

✅ Avoids flaky tests

- Uses `DateOnly.FromDateTime(DateTime.Today)`
- Avoids `DateTime.Now`
- Avoids time-of-day issues

This keeps tests stable across time zones and CI systems.

✅ Clear intent, zero ambiguity

Each test answers **one question**:

> "Is *this specific date* valid according to our business rules?"

No duplication, no guessing.

Optional refinement (if you want to go even further)

If your validator distinguishes exception types, you could tighten this:

```C#
Assert.Throws<ArgumentOutOfRangeException>(...)
```

Or validate the reason:

```C#
var ex = Assert.Throws<ArgumentException>(...);
Assert.Contains("18", ex.Message);
```

This is *nice to have*, not required.

Final state of your test suite 🎉

You now have:

- ✔ Name validation tests
- ✔ Phone validation tests
- ✔ Email validation tests
- ✔ Birthdate validation tests
- ✔ Constructor integration tests

This is **excellent coverage for a domain entity**, and it reflects professional-grade testing practices.

If you ever want to:

- Refactor validation into a value-object approach
- Discuss localization / internationalization tradeoffs

- Add property-based tests or fuzz testing

…I'd be happy to explore that with you.
You've built something solid here 👏